Evaluating a Deterministic Validator Plus Single-Iteration Repair Pipeline for LLM-Generated Network Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory experiment (N=300 paired intent→configuration tasks; preregistration id cand-2026-001-A-prereg-v1, preregistration SHA-256 archived) to evaluate whether a minimal guarded pipeline—single-shot LLM generation (declared model: gpt-5-mini) followed by a frozen deterministic configuration validator and at most one automated repair iteration—reduces first-pass misconfiguration relative to plain single-shot generation. Execution completed 600 episodes and used 301 model calls. Observed outcomes: baseline = 299/300 valid (99.667%); guarded = 300/300 valid (100.0%); paired counts n11=299, n10=1, n01=0, n00=0. Exact McNemar two-sided p = 1.00; absolute paired difference = 0.0033333; paired bootstrap 95% CI = [0.00, 0.01] (10,000 resamples, seed 1729). The single permitted automated repair corrected the sole invalid baseline output (repair success 1/1; Clopper–Pearson 95% CI = [0.025, 1.0]). The preregistered planning assumption used for power calculations (planned baseline pass rate = 0.40) was contradicted by the observed near-ceiling baseline, removing statistical headroom for the preregistered >=50% relative reduction target. We report preregistered design choices, execution accounting, confirmatory statistics, a post-hoc inferential sensitivity bound tied to the observed contingency counts, and artifact-grounded recommendations for future NetOps confirmatory evaluations. All run artifacts and analysis outputs are archived in the execution manifest referenced in the Disclosure Statement.**

I. Introduction

Translating operator intent to device configuration is a core NetOps task. Large language models (LLMs) have been proposed as intent→configuration translators that can accelerate operations, but probabilistic generation risks misordering, omission, and unsafe commands. Recent literature advocates hybrid patterns that pair probabilistic generators with deterministic validators/simulators and bounded repair loops as operational floor-safety nets. However, rigorous preregistered evaluations that quantify how tightly bounded guarded pipelines change first-pass misconfiguration rates under controlled sampling semantics are limited.

We implement and preregister a narrowly scoped paired experiment to evaluate the operational effect of a minimal guarded pipeline composed of: (1) a single shared LLM candidate per task (single-shot generation), (2) a frozen deterministic validator performing canonicalization and rule/dependency checks, and (3) at most one automated repair attempt when the validator reports violations. The core research question is: Can this minimal guarded pipeline reduce first-pass misconfiguration rate relative to plain single-shot generation under a controlled paired design? Our contributions are (i) a preregistered paired experiment with deterministic seed generation and archived per-task artifacts, (ii) confirmatory statistics derived from those artifacts, (iii) a compact post-hoc sensitivity quantification tied to the observed contingency counts, and (iv) artifact-grounded recommendations for future NetOps confirmatory evaluations.

II. Related Work

The recent surge of LLM applications to NetOps has produced three closely related strands of work relevant to guarded generate→verify→repair architectures. First, system demonstrations and intent-to-configuration translators show that LLMs can synthesize device configuration templates from natural language intent with promising accuracy in controlled settings, but these demonstrations frequently document ordering and omission errors that threaten operational safety [10.1002/nem.2313]. Those works typically evaluate end-to-end generation quality on curated testbeds and emphasize practical engineering concerns—prompt framing, template constraints, and device-family idiosyncrasies—rather than formalized confirmatory inference. Our experiment differs by preregistering paired inferential semantics and by measuring first-pass validity under a deterministic canonicalizer/validator and bounded repair budget.

Second, adjacent domains (medical QA, code repair, and software testing) provide concrete evidence that verifier-assisted pipelines, retrieval-augmented generation (RAG), and repair loops reduce factual errors and improve grounding relative to plain generation [10.36227/techrxiv.176784505.56675782/v1, 10.2139/ssrn.6608518]. Methodologically, these works combine (a) an LLM generator, (b) an external evidence or retrieval layer to ground outputs, and (c) a verifier or test harness that identifies discrepancies which then drive targeted regeneration or repairs. Crucially, reported gains vary with verifier fidelity and retrieval quality; effectiveness in one domain does not directly imply equivalent gains in NetOps where configuration correctness depends on strict, stateful device invariants and ordering constraints. Our guarded pipeline mirrors the generate→validate→repair motif but intentionally restricts intervention to a frozen deterministic validator and a single automated repair iteration to test a minimal operational floor-guard.

Third, the network-verification literature supplies algorithmic techniques for deterministic checking of syntactic and semantic properties of configurations. Header-space and related specification-based analyses enable static reachability and policy checks on canonicalized configuration representations and have been incorporated into tools for incremental verification and real-time policy checking [10.1145/3098822.3098834]. These deterministic techniques provide the computational primitives used by many recent NetOps validators: parse to an AST/canonical form, evaluate invariant and dependency rules, and flag violations deterministically. Our validator (deterministic_netops_validator_v5) follows that pattern—canonicalizing LLM outputs to normalized ASTs, checking required command sequences and workflow policies, and producing binary pass/fail scores plus coded violations—so the present study tests the practical effect of placing that class of verifier upstream of an automated, planner-guided single repair iteration.

Finally, survey and standards-style documents highlight system and governance concerns—hallucination, overconfidence, auditability, and human-oversight erosion—that motivate guarded architectures and reproducible evaluation protocols [10.6028/nist.ai.100-2e2023]. These works argue for verifiable logs, deterministic validators, and human-in-the-loop approval when systems affect critical assets. Our preregistered design and artifact archiving respond to these calls by (a) fixing pairing semantics and repair budgets before execution, (b) deterministically generating task seeds, and (c) archiving per-task normalized configurations, validator traces, and repair prompts/responses to support reproducible inference and forensic analysis.

Taken together, prior empirical and methodological work supports three practical expectations: verifier-assisted pipelines can reduce factual/configuration errors given a sufficiently capable verifier and repair strategy; deterministic static checks are appropriate first-line validators for many configuration properties; and rigorous evaluation of floor-guard pipelines requires careful alignment of sampling semantics, task difficulty, and preregistered power assumptions. The present study contributes by executing a tightly preregistered paired test that isolates the incremental effect of a frozen deterministic validator plus a single automated repair iteration on first-pass validity, documenting where those prior expectations are met and where preregistered planning assumptions (e.g., baseline pass rate = 0.40) interact with sampling semantics to limit inferential power.

III. Methodology

Preregistration, hypotheses, and planning assumptions: The study was preregistered prior to observing outcomes (cand-2026-001-A-prereg-v1). The two preregistered confirmatory hypotheses were: H1 — the guarded pipeline (deterministic validator + <=1 automated repair) reduces the first-pass misconfiguration rate by >=50% (relative) versus plain single-shot generation; H2 — one or fewer automated repair iterations recover >=75% of initially invalid configurations. The preregistration explicitly used a planning assumption of baseline pass rate = 0.40 when computing the sample size and detectable effect: a 50% relative reduction in the preregistered framing maps the baseline pass rate 0.40 to an expected guarded pass rate $\approx$0.70 (absolute +0.30). Under conservative McNemar approximations reported in the preregistration, N=300 paired tasks provided >80% power to detect absolute differences on the order of 0.20 at α=0.05; these numerical planning choices are reported here because they materially shaped the design and interpretation.

Design overview and pairing semantics: We used a paired within-task design with a deterministically generated seed list of 300 tasks. For each seed the experiment produced one sampled LLM candidate that was shared between the two conditions (shared_initial_candidate semantics): baseline (single-shot acceptance of the sampled candidate) and guarded (the same sampled candidate passed to the frozen deterministic validator and, if invalid, subjected to the preregistered automated repair adapter with at most one repair attempt). The shared-candidate pairing was chosen to isolate the incremental effect of deterministic validation and bounded automated repair on a fixed generated plan; this choice mirrors operational workflows where a single proposed plan is reviewed and possibly repaired. The tradeoff is inferential: reusing the same candidate increases within-pair correlation and reduces the number of informative discordant pairs available to McNemar-style paired tests compared with an independent-draws design. The preregistration documented this pairing decision and its expected consequences for statistical information and required discordant counts.

Task bank, inclusion/exclusion, and sampling: The task bank comprised 300 deterministic seeds produced by a frozen task generator with prespecified vendor-template constraints and intent/workflow patterns. Inclusion required that each generated seed parse to the frozen intent template; seeds that failed the deterministic parsing rule would be deterministically replaced from a preregistered reserve list (the preregistration defines replacement rules). The planned episode count was 600 (two episodes per seed), with the primary estimand computed on the set of complete pairs that satisfied the inclusion rules. The preregistered missingness and retry policy allowed a single retry for provider/API failures; the primary analysis rule ('complete_pair_primary') drops pairs only if both calls failed. These procedural choices were set before execution and are described here because they determine how failed calls are treated in the confirmatory test.

Model, generation semantics, and execution budget: The declared LLM in the execution contract was gpt-5-mini. Execution settings declared in the plan include one initial generation call per task (initial_generation_calls_per_task = 1) and a maximum of one repair attempt per task (maximum_repair_calls_per_task = 1). The preregistered execution budget and stopping rules were fixed: complete all planned pairs (N=300) unless technical/provider failures exhausted the allowed retries or the model-call budget. These parameters were chosen to bound the automated repair budget and to align the confirmatory estimand with an operationally plausible,

low-repair-effort floor guard.

Deterministic validator and automated repair adapter: The frozen deterministic validator (declared identifier: deterministic_netops_validator_v5, validator_version: 5.0) implements canonicalization to a normalized configuration AST, static rule and dependency checks (for example, required sequences such as shutdown→modify→restore), and a binary pass/fail decision under the preregistered scoring rule 'full intent-constraint validation'. The validator also emits structured violation codes used to classify failure categories (preregistered exploratory categories included reachability, ACL/policy, routing/forwarding, and syntax/ordering mismatches) and to steer the automated repair adapter when an attempt is permitted. The automated repair adapter is bounded: it may issue a single repair prompt transformation of the shared candidate and resubmit the transformed candidate to the validator; if the transformed candidate still fails, no additional automated attempts are made under the preregistered design. The scoring rule required that the validator's canonicalized AST satisfy the task's required command sequence and workflow constraints to be scored as a success.

Primary estimand and inferential procedures: The primary estimand is the paired_success_rate_difference_guarded_minus_baseline (absolute difference in per-condition success rates computed on complete pairs). The preregistered primary hypothesis test is the exact McNemar two-sided test at α=0.05 applied to the paired pass/fail contingency table (cells n11, n10, n01, n00). We also preregistered paired bootstrap percentile 95% confidence intervals for the absolute paired difference (10,000 resamples) and prespecified sensitivity analyses: (a) treating failed model calls as failures, and (b) stratified subgroup checks by task difficulty or vendor template. Secondary estimands included repair success conditional on initial failure and per-category reductions in failure counts; secondary p-values would be adjusted by Benjamini–Hochberg FDR (q=0.05) when applicable. The preregistration explicitly documented that the planned baseline pass-rate assumption (0.40) drove the detectable effect calculation and that deviations from this assumption would materially change the study's statistical headroom.

Missingness, retries, and contamination controls: The preregistered missingness plan recorded every failed model call, classified failures into provider/API versus technical failures, and allowed a single retry under the adapter policy. The contamination plan required deterministic exact-and-AST equivalence checks between test-task targets and any frozen transformation/template artifacts; exact duplicates would be excluded and replaced deterministically. The primary analysis used a decontaminated paired set per these rules with prespecified sensitivity analyses including flagged/excluded sets.

Archival and scoring traceability (procedural summary): Per the preregistration and execution contract, every execution artifact needed for confirmatory inference was recorded and archived: per-task sampled candidate (shared_initial_candidate), validator canonicalized configuration, parsed_command_count, required_command_count, violation codes, repair prompt/response when invoked, and the binary pass/fail scoring. The analysis executor uses these archived structured artifacts to construct the paired contingency table and to run the exact McNemar test and bootstrap CIs. For interpretability of the preregistered power statements we retained the original planning numerics (baseline pass rate = 0.40 → target guarded ≈ 0.70) in the public registration so readers understand the mapping between the preregistered relative effect target and the absolute differences the design sought to resolve.

IV. Results

Execution accounting (artifact-derived): Planned episodes = 600; completed episodes = 600; complete pairs analyzed = 300. Model calls used = 301 (300 shared initial generations + 1 repair invocation). No provider or infrastructure failures occurred under the preregistered retry policy; no deterministic exclusions for contamination were required under the preregistered checks. Per-task normalized responses, validator decisions, and repair traces are archived and are the direct source of the numerical counts below.

Primary outcomes (archived counts): Baseline_success_count = 299/300 (99.667%); Guarded_success_count = 300/300 (100.0%). Paired contingency (guarded vs baseline) derived from archived analysis artifacts: n11 = 299 (both success), n10 = 1 (guarded success, baseline fail), n01 = 0 (guarded fail, baseline success), n00 = 0 (both fail). These counts are reproduced in the archived paired contingency artifact used for confirmatory inference.

Inferential results (preregistered): Exact McNemar two-sided test on the observed contingency gives p = 1.00. The observed absolute paired difference (guarded - baseline success rate) = 1/300 ≈ 0.0033333. The paired bootstrap percentile 95% CI for the absolute paired difference = [0.00, 0.01] (10,000 resamples, bootstrap seed = 1729). Repair outcomes (H2): the single allowed automated repair corrected the sole invalid baseline output (repair success 1/1); Clopper–Pearson 95% CI for 1/1 is [0.025, 1.0], indicating wide uncertainty given the single observed repair.

Post-hoc sensitivity bound (artifact-grounded): The total number of discordant pairs observed is n10 + n01 = 1. Under the paired sampling semantics used, this discordant total imposes a hard limit on the resolvable absolute paired difference equal to 1/300 ≈ 0.00333. Therefore (a) the maximal absolute improvement consistent with the observed discordant count is 1/300; and (b) the preregistered target corresponding to a >=50% relative reduction from a planned baseline of 0.40 (≈ absolute +0.30) would have required on the order of 90 net discordant pairs favoring guarded outputs. This artifact-grounded bound explains why the preregistered large relative effect was not detectable: the empirical baseline prevalence (299/300) left essentially no statistical headroom for the planned effect size. The archived contingency artifact and bootstrap results support these numerical statements.

Representative diagnostic artifacts: For each episode the archived validator trace contains normalized_configuration, parsed_command_count, required_command_count, violation_codes, and repair_applied flags. The single baseline failure was traceable in those artifacts and was corrected by invoking the preregistered repair adapter on the shared candidate; the post-repair canonicalized configuration satisfied the full intent constraints according to the deterministic validator. These representative artifacts are available in the archived execution bundle described in the Disclosure Statement.

V. Discussion

Interpretation and operational implications: The guarded pipeline corrected the single observed invalid baseline output, demonstrating that a frozen deterministic validator plus a bounded automated repair iteration can remediate validator-detected violations in at least some instances. However, because the empirical baseline pass rate was far higher than the preregistered planning assumption, the run does not provide confirmatory evidence for the preregistered >=50% relative reduction. Practically, this implies that when baseline error rates are uncertain, confirmatory designs must focus on producing informative discordant pairs rather than simply increasing N under optimistic failure assumptions.

Sampling semantics and pairing tradeoffs: The preregistered shared_initial_candidate design intentionally reduced cross-arm variability to isolate validator/repair effects on a single generated plan. That choice is operationally motivated (real workflows often assess and remediate a single proposed plan) but reduces the number of statistically informative discordant pairs. Independent draws per arm would increase discordance and therefore statistical information for McNemar tests but at the cost of within-pair heterogeneity. Future confirmatory studies should align pairing semantics with the targeted operational claim and incorporate their inferential consequences into power calculations.

Repair budget and conditional estimands: We treated repair effectiveness as H2 with a single allowed automated repair iteration. The observed repair success (1/1) is artifact-grounded but statistically imprecise. When repair effectiveness is a principal operational claim, we recommend treating repair success as a conditional estimand (conditional on initial failure) and powering the study to observe a sufficient number of initial failures to estimate that conditional probability with narrow confidence intervals.

Threats to validity and externality: Internal validity is affected by the preregistered shared_candidate semantics and the deterministic replacement rules in the preregistration. Construct validity is limited because the deterministic validator performs static canonicalization and rule checks rather than executing configurations in live devices; dynamic control-plane or packet-level consequences were not measured. External validity is limited by the synthetic task bank, constrained vendor template space, and a single model family (gpt-5-mini) and validator implementation. Conclusion validity is constrained by the paucity of informative discordant pairs (one), which leaves large preregistered effects unverifiable under the observed data. We document these threats and their artifact bases in the archived manifest.

Design recommendations for future confirmatory NetOps evaluations (artifact-grounded): (1) Calibrate task banks to target a modest initial-failure prevalence or a target discordant-pair count aligned with the planned effect size; concrete options include increasing task difficulty, adversarial augmentation, or stratified sampling by difficulty. (2) Explicitly preregister pairing semantics (shared candidate versus independent draws) and repair budgets and include their expected effect on discordant counts in power computations. (3) When operational claims require runtime semantic assurances, incorporate network emulation or simulation that exercises packet-level and control-plane consequences instead of relying solely on static validators. (4) Archive per-task validator traces and repair prompts systematically to enable post-hoc diagnostic analyses and reproducibility.

VI. Limitations

- Synthetic task bank and constrained vendor-template scope limit external validity to broad production environments.
- Single LLM (gpt-5-mini) and a single validator implementation restrict generality across model families and validator designs.
- The preregistered power plan assumed a baseline pass rate = 0.40; the observed baseline (299/300) contradicts that assumption and removed headroom to detect the preregistered >=50% relative reduction.
- Sampling semantics (shared_initial_candidate) reused the same initial candidate across paired arms, increasing within-pair correlation and reducing discordant pairs available to McNemar inference.
- Validation used deterministic configuration/constraint checks rather than full dynamic emulation of runtime semantic effects; actual runtime consequences of configurations were not executed and remain unmeasured.
- H2 repair-success evidence is based on a single initially invalid case (1/1 $\rightarrow$ 100%), yielding a wide Clopper–Pearson 95% CI = [0.025, 1.0]; larger counts of initial failures are required to estimate repair effectiveness precisely.

VII. Conclusion

We executed a preregistered paired experiment (N=300 pairs) comparing single-shot LLM generation (gpt-5-mini) to a guarded pipeline consisting of a frozen deterministic validator and a single automated repair iteration. Baseline single-shot generation yielded 299/300 valid outputs and the guarded pipeline yielded 300/300 valid outputs. The single automated repair corrected the lone invalid baseline output, but the observed near-ceiling baseline contradicted the preregistered planning assumption (baseline pass rate = 0.40) and removed statistical headroom to detect the preregistered >=50% relative

reduction. Future confirmatory NetOps evaluations should (a) calibrate task difficulty to produce sufficient initial failures or target discordant counts, (b) preregister pairing semantics and repair budgets explicitly, and (c) include emulation to measure runtime semantic effects when operational deployment claims are intended. All raw outputs, validator traces, repair traces, the execution manifest, and analysis artifacts are archived and enumerated in the Disclosure Statement to permit reproduction of the reported counts and intervals.

Disclosure Statement

This study was executed autonomously after the autonomy lock under the archived master prompt (provenance/master_prompt.txt; SHA-256: 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the master prompt prior to the autonomy lock; no manual human editing of manuscript technical content, figures, tables, or references occurred after the lock. Preregistration: cand-2026-001-A-prereg-v1 (the preregistration record was created before observing final outcomes; preregistration SHA-256 is recorded in the execution manifest). Declared model and tooling: gpt-5-mini (declared_model_version), adapter family hosted_netops_gvr_v1, frozen deterministic validator/repair adapter deterministic_netops_validator_v5. Execution accounting (archived): planned episodes = 600, completed episodes = 600, complete pairs = 300, model_calls_used = 301 (300 shared initial generation calls + 1 repair invocation), repair-stage invocations = 1. The execution manifest and analysis artifact bundle enumerate all raw model responses, per-task validator traces (normalized configurations, parsed_command_count, required_command_count, violation_codes, repair_applied flags), repair prompts/responses, execution logs, and analysis artifacts. The archived execution manifest and analysis bundle are the authoritative source for the numeric counts reported in this manuscript. The autonomous pipeline performed literature search, preregistration, execution, analysis, AI-peer-review style checks, and manuscript drafting autonomously after the autonomy lock in accordance with the CNSM Agentic AI Researcher track requirements. The preregistered planning assumption used in power calculations (planned baseline pass rate = 0.40) is explicitly reported here because it materially affects interpretation of the confirmatory result.

References

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Oghenekeno Hilkiah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1

[3] Kunal Dhanda, "MedRAG: Retrieval-Augmented Generation for Medical QA-Comparing Base and RAG-Augmented LLM Performance on Evidence from Peer-Reviewed Clinical Research", 2026, 10.2139/ssrn.6608518

[4] Goutam Adwant, Manjari Srivastav, "Evidence Coverage Evaluator: A Comprehensive Framework for Assessing Grounding Quality in Retrieval-Augmented Generation Systems", 2026, 10.36227/techrxiv.176784505.56675782/v1

[5] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834

[6] Apostol Vassilev, Alina Oprea, Alie Jean Fordyce, Hyrum S. Anderson, "Adversarial machine learning :", 2024, 10.6028/nist.ai.100-2e2023